



Network Architecture Lab - Semester Project

Project Final Report

Author:
Sheng-Wen Chen [414932]

June 2026

Contents

1	Motivation	3
2	Background	3
2.1	Token-Bucket Policing and TCP Windows	3
2.2	Prior Work in ns-3	3
2.3	What Changes in a Physical Linux Setup	4
3	Experimental Setup	4
3.1	Physical Topology and First Measurement Attempt	4
3.2	Custom Congestion-Control Module	5
3.3	Handling Kernel Ring-Buffer Overwrite	6
3.4	Final Automation Pipeline	7
3.5	Receiver-Side Window Clamp	9
4	Experiment 1: Sender-Side CWND Capping	9
4.1	Parameter Sweep	9
4.2	Empirical Cap Selection	9
4.3	Early Sanity Check: Large Queue	10
4.4	Sweep Results	10
4.5	Why the Original Formula Overestimated	12
4.6	Remaining Model Mismatch	13
5	Experiment 2: Receiver-Side RWND Control	13
5.1	Motivation	13
5.2	Method	14
5.3	Results	14
6	Experiment 3: Estimating the Token Generation Rate	14
6.1	Relation to Policing-Rate Inference	14
6.2	Cumulative Arrival at the Receiver	15
6.3	Receiver-Assisted Enqueue-Rate Estimation	15
7	Limitations	15
8	Conclusion	16

1. Motivation

Internet Service Providers often use rate limiters to enforce a maximum traffic rate for a subscriber, an application class, or a traffic aggregate. A common implementation is a Token Bucket Filter (TBF): tokens are generated at a configured rate, packets consume tokens before transmission, and excess packets are either delayed in a queue (traffic shaping) or dropped when the queue is full (traffic policing). This creates a mismatch with loss-based TCP congestion control. TCP Reno and similar congestion-control algorithms grow `cwnd` until loss and then reduce it. Behind a per-flow limiter, this probing can prevent the connection from using the full allowed rate smoothly: the sender repeatedly overshoots the limiter, fills the queue, loses packets, and enters recovery. The central question is therefore: **if the limiter parameters are known, or can be inferred, can the endpoint choose a better sending window and use the available rate with less loss?**

Two previous semester projects studied this question in simulation. Friedli's work used ns-3 to test whether knowing the TBF parameters could improve TCP throughput by selecting a better window. The important result for this project is that the gain is largest when the queue is small; in that regime, the fixed-window algorithms improved throughput by about 33% for New Reno. Rimkevicius's work studied the complementary problem: how to infer the policing rate from packet arrivals, especially from the receiver side.

However, both works were evaluated in a simulated environment, where the bottleneck, queue size, RTT, and packet losses are easy to control and observe. This project asks whether the same idea survives in a real physical setup. We built a two-machine Linux testbed connected through a lab router, added a TBF on the server-to-client path, instrumented the Linux TCP stack, and compared uncapped Reno-like behavior with capped windows. The resulting story is not simply that the formulas work or fail. Window capping does help strongly in the small-queue regime, matching the motivation from Friedli's result, but the physical setup exposes an additional effect: the server can enqueue bursts into the TBF faster than the TBF drains them. That enqueue pressure reduces the amount of queue space that can be treated as useful extra in-flight capacity.

2. Background

2.1. Token-Bucket Policing and TCP Windows

A TBF is described by three parameters: a token generation rate R_{tbf} , a burst size B , and a queue limit Q_{config} . The token rate determines the long-term output rate. The burst size determines how many tokens can accumulate when the sender is idle or sending below the token rate. The queue limit determines how much traffic can wait for tokens before packets are dropped.

TCP controls sending through two windows. The congestion window (`cwnd`) is chosen by the congestion-control algorithm at the sender. The receive window (`rwnd`) is advertised by the receiver and expresses how much data the receiver is willing to accept. The effective sending window is approximately:

$$W_{\text{send}} = \min(\text{cwnd}, \text{rwnd}).$$

Traditional loss-based algorithms such as TCP Reno and TCP Cubic grow `cwnd` until loss, then reduce it. This behavior is useful when loss indicates shared-network congestion, but it can be inefficient behind a per-flow TBF. If the user has a dedicated rate limit, the best behavior is often to send steadily at the token rate rather than probing above it and recovering from avoidable drops.

2.2. Prior Work in ns-3

Friedli's project, *Revisiting Traffic Congestion Control Protocols at the Last Mile*, studied how TCP New Reno, Cubic, and BBR interact with a TBF in ns-3. The key observation was that the maximum useful

amount of in-flight data is related to the BDP and to the bottleneck queue. The project proposed:

$$W_1 = \text{BDP} + Q_{\text{config}} - \text{MSS}$$

and later found that a simpler cap,

$$W_2 = \text{BDP} + \text{MSS},$$

also reaches the TBF rate in many configurations while avoiding buffer build-up. In simulation, **the algorithms improved small-buffer cases by roughly 33% for New Reno and 17% for Cubic**. For larger queues or larger bursts, default TCP already performs closer to the TBF rate, so the relative gain decreases.

Rimkevicius's project, *Inference of Traffic Differentiation Policing Rate*, studied how to infer R_{tbf} from one traffic trace. It compared sender-side and receiver-side methods. The most accurate method was **receiver-side cumulative-arrival differentiation**: compute cumulative received bytes over time, differentiate the curve, and use a statistic such as the median or a lower percentile of the derivative samples. Loss-based methods were more conservative, but often underestimated the true policing rate. This project used that work as the natural path toward a receiver-driven system: first infer R_{tbf} , then choose a window cap from the inferred rate.

2.3. What Changes in a Physical Linux Setup

The ns-3 model hides several details that matter in Linux. In Linux, packets normally pass through a queueing discipline, or qdisc, before they are handed to the network driver. A TBF is one such qdisc: it releases packets only when enough tokens are available, and otherwise keeps them in a bounded waiting queue. If that queue reaches the configured `limit`, new packets cannot simply wait forever and are dropped [6, 7]. It is straightforward to attach TBF to the server's egress path, so the server-to-client traffic is shaped before leaving the server. Applying the same mechanism after traffic has already arrived at the client would require a more complex ingress setup, such as redirecting packets through an `ifb` virtual interface. For reproducibility, the stable experiments therefore placed the TBF on server egress, which emulates a forward-path bottleneck while keeping the limiter parameters directly under our control.

Second, the sender does not place one MTU-sized packet at a time into the qdisc. With TCP Segmentation Offload (TSO) and Generic Segmentation Offload (GSO), the kernel or NIC may handle large segments that are later split into MTU-sized packets. This produced large qdisc enqueue events and NIC requeues in early experiments, making the measured `cwnd` behavior difficult to interpret. The final automation disables TSO and GSO before applying the TBF.

Third, measuring `cwnd` at high frequency is itself difficult. Polling with `ss` cannot keep up with fast changes because it requires repeated Netlink dumps. Kernel logging with `pr_info` gives better visibility, but Linux uses a finite ring buffer, so frequent logs are overwritten unless they are extracted and cleared during the transfer.

3. Experimental Setup

3.1. Physical Topology and First Measurement Attempt

The physical setup consists of a Linux server, a lab router, and a Linux client. The data direction of interest is:

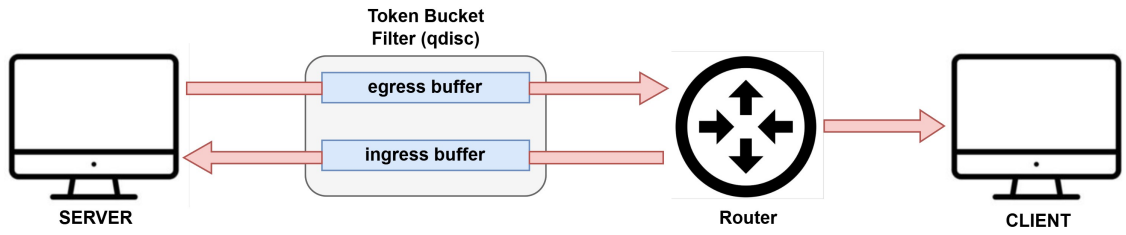


Figure 1: Physical setup used in the experiments. The TBF is applied on the server egress path, so traffic is shaped before it reaches the router and client.

The server sends a controlled **bulk transfer** to the client. The TBF is installed on the server egress interface, so the server-to-client traffic sees a reproducible token-bucket bottleneck before reaching the router.

The first measurement problem was extracting `cwnd`. Initially, `ss` was polled from user space to observe the TCP socket state. This was not sufficient: the `cwnd` transitions during slow start and recovery happen faster than the polling loop can reliably capture. As a result, the trace was too sparse to identify the Reno sawtooth or to choose a window cap (Figure 2). The setup therefore moved from user-space polling to kernel-side instrumentation.

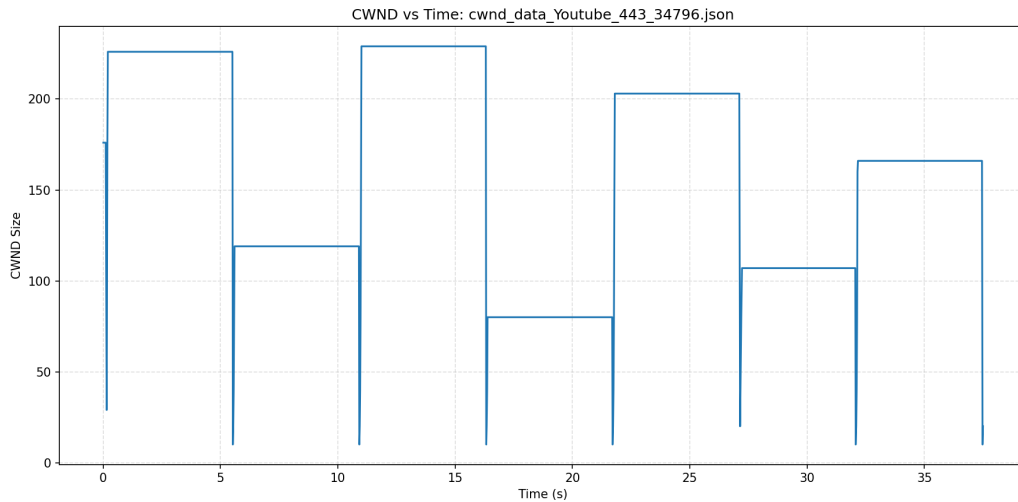


Figure 2: Early `ss`-based `cwnd` trace. The trace misses fast transitions, so it is not reliable enough for selecting a window cap.

3.2. Custom Congestion-Control Module

The project then moved to a controlled bulk-transfer setup and implemented a custom Linux congestion-control module, `my_cca`. This did not require editing and recompiling the whole Linux kernel. Instead, the project implemented a loadable TCP congestion-control module that registers itself with Linux's pluggable TCP congestion-control framework. This framework exposes a `struct tcp_congestion_ops`, where a module provides callbacks such as `ssthresh`, `cong_avoid`, `undo_cwnd`, and `pkts_acked` [3]. Figure 3 shows the corresponding registration and logging code used by `my_cca`. The module is Reno-style: it implements slow start, additive increase, Reno-like slow-start threshold reduction, and an undo callback. The important modification is a module parameter, `cwnd_limit`. When this parameter is nonzero, both slow start and congestion avoidance apply:

```
cwnd ← min(cwnd, cwnd_limit).
```

When `cwnd_limit = 0`, the cap is disabled and the module behaves as an uncapped Reno-like baseline. This made it possible to compare capped and uncapped behavior while keeping the same logging and analysis pipeline.

The module logs `cwnd`, RTT samples, bytes acknowledged, bytes sent, and a phase label whenever enough ACKs have passed or the phase changes. The phase label is derived from the Linux TCP congestion-control state first, and from the slow-start condition only if the socket is not already in a recovery state. In other words, the labels are assigned in the same priority order as the code:

- **fast_retransmit**: `icsk_ca_state` is `TCP_CA_Recovery`. This is the fast-recovery state entered after loss is inferred from duplicate ACKs or SACK information.
- **loss_recovery**: `icsk_ca_state` is `TCP_CA_Loss` or `TCP_CA_CWR`. `TCP_CA_Loss` usually follows an RTO timeout, while `TCP_CA_CWR` means TCP is reducing `cwnd` after a congestion signal such as ECN or another congestion-window reduction event. This label therefore means “loss or congestion-window reduction”, not necessarily a directly observed packet loss.
- **slow_start**: the socket is not in `TCP_CA_Recovery`, `TCP_CA_Loss`, or `TCP_CA_CWR`, and `tcp_in_slow_start(tp)` is true. In practice, this means `cwnd` is still below `ssthresh`.
- **congestion_avoidance**: none of the previous conditions hold. This is the normal Reno additive-increase phase after slow start.

This ordering is important when reading the plots. A point is labeled as a recovery phase before the code checks whether it is below `ssthresh`; only non-recovery points can become `slow_start`. Therefore, if a loss event reduces `cwnd` below the initial value or below `ssthresh`, the following samples may still be valid TCP behavior rather than an instrumentation error.

The module uses `pr_info()` inside the ACK callback to log TCP state from the kernel. Logging every ACK was too expensive and quickly overwrote the kernel ring buffer. The final version logs every 100th ACK unless the TCP phase changes. A sequence number was added to detect missing log records.

3.3. Handling Kernel Ring-Buffer Overwrite

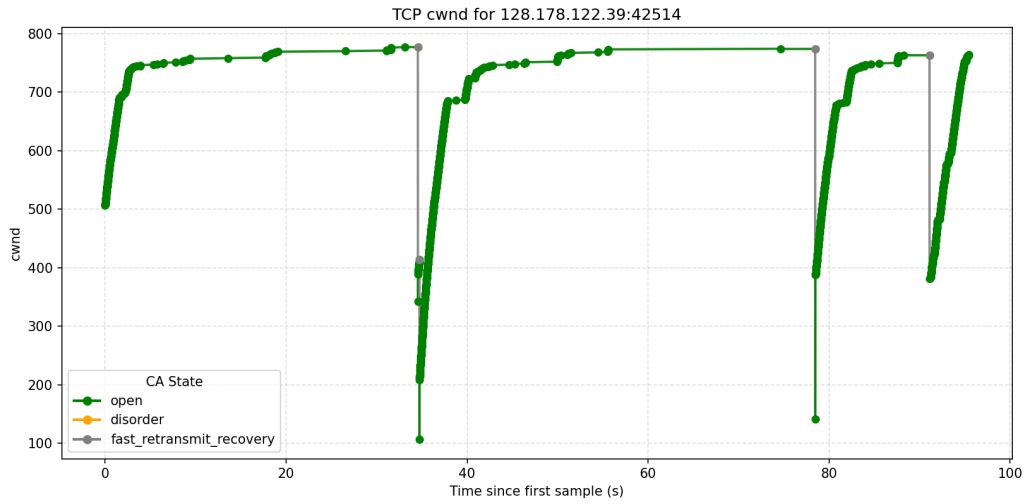
The first custom-module runs still produced misleading traces: for example, a 20 GiB transfer did not appear to start from the expected initial `cwnd`: 10. The cause was not TCP behavior, but instrumentation loss. By the time logs were extracted at the end of a transfer, early `pr_info` records had already been overwritten in the **kernel ring buffer**.

In Linux kernel code, `printk()` and its convenience macros such as `pr_info()` are the standard way to write diagnostic messages from inside the kernel. These messages go into the kernel log buffer, a finite ring buffer exported to user space through `/dev/kmsg`. The `dmesg` command reads this buffer, and `dmesg -clear` clears the records that have already been read [9, 8]. In our module, each selected ACK event calls `pr_info()` with the current `cwnd`, RTT sample, bytes acknowledged, bytes sent, TCP phase, destination IP, and destination port, as shown in Figure 3. Because the log buffer has finite capacity, high-frequency `pr_info()` calls can overwrite older records before the user-space analysis script reads them. This is why the early experiments appeared to miss the beginning of slow start.



Figure 3: Implementation snippets for the custom congestion-control module.

The final server therefore starts a background log extractor. It repeatedly reads `dmesg`, appends relevant kernel logs to a context file, clears the ring buffer, and sleeps for a configurable interval. This allowed the project to capture the beginning of the transfer and the phase transitions more reliably. Figure 4 shows the type of broken trace observed before this fix.

Figure 4: Early custom-CCA trace before reliable ring-buffer extraction. Missing early records made the `cwnd` evolution look inconsistent.

3.4. Final Automation Pipeline

After the measurement path was reliable, the setup was wrapped into an automation pipeline. Figure 5 shows the final system. The pipeline has two entry points. `client_server/run.sh` performs one complete experiment on the server host, while `client_server/run_cwnd_sweep.sh` repeatedly calls `run.sh`, starts the client over SSH, and aggregates the sweep outputs.

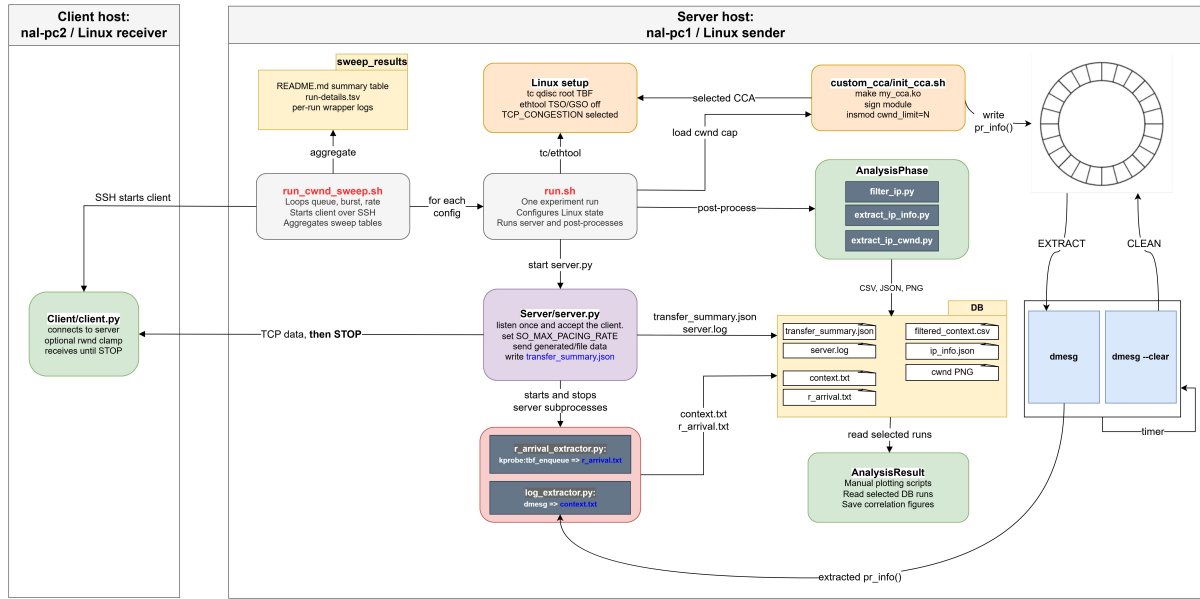


Figure 5: Final automation pipeline. **run.sh** prepares the Linux sender, starts the instrumented server, collects raw logs, and launches the per-run analysis. **run_cwnd_sweep.sh** wraps this process across many TBF and window configurations.

For a single run, **run.sh** turns user-specified parameters such as **-cca**, **-size-gib**, **-cwnd**, **-tbf-rate**, **-tbf-burst**, and **-tbf-limit** into a reproducible measurement:

1. parse and validate the experiment options and required tools;
2. create a temporary output directory under **DB/**;
3. reset the root qdisc on the selected server interface;
4. install the TBF qdisc with the chosen token rate, burst size, and queue limit;
5. disable TSO and GSO with **ethtool**, so packet and qdisc behavior are easier to interpret;
6. if **my_cca** is selected, rebuild, sign, unload, and reload the module with the requested **cwnd_limit**;
7. start **Server/server.py** with the selected TCP congestion-control algorithm and the instrumentation flags;
8. wait for one client connection, send the configured amount of data, and record the transfer summary;
9. move the temporary artifacts into a permanent per-run directory under **DB/**;
10. for **my_cca** runs, filter the kernel logs for the specific client IP and ephemeral port, then generate the analysis CSV, JSON summary, and **cwnd** plot.

The Python server is responsible for the live transfer and for starting the instrumentation subprocesses. The log extractor repeatedly reads **dmesg**, appends matching kernel messages to **context.txt**, and clears the kernel ring buffer. The arrival-rate extractor uses a **bpftrace** probe on **tb_enqueue** to sample bytes entering the TBF enqueue path and writes **r_arrival.txt**. The server itself accepts one client, sets the socket's congestion-control algorithm, sets **SO_MAX_PACING_RATE** to 1 Gbit/s by default, sends generated data, sends **STOP**, and records transfer metadata.

The output of one run is therefore both raw and processed. The raw files, including **server.log**, **transfer_summary.json**, **context.txt**, and **r_arrival.txt**, preserve what happened during the transfer. The processed files, including **filtered_context.csv**, **ip_info.json**, and the generated **cwnd** plot,

are the analysis artifacts used in the figures and tables. The sweep wrapper adds another layer above this: for each queue, burst, and token-generation rate, it runs the uncapped baseline, searches for an empirical `cwnd`, evaluates the fixed BDP-based formulas, and writes aggregate results under `sweep_results/`.

Unless otherwise stated, the experiments use 1 GiB transfers, a base RTT around 0.4 ms, and MSS 1460 B. The physical topology is an isolated local network with the server and client in the same subnet. The intended bottleneck is therefore the server egress TBF, not the router or the wider Internet.

3.5. Receiver-Side Window Clamp

The sender-side cap is useful for experimentation, but a realistic client cannot normally install a custom congestion-control module on a remote server. Therefore the project also tested whether the same behavior can be induced from the client by limiting `rwnd`. The Python client uses:

$$\text{SO_RCVBUF} = \text{TCP_WINDOW_CLAMP} = W_{\text{target}}.$$

Since the sender transmits according to $\min(\text{cwnd}, \text{rwnd})$, advertising a smaller receive window should constrain the sender even when the server-side `cwnd` cap is disabled.

4. Experiment 1: Sender-Side CWND Capping

4.1. Parameter Sweep

The main sweep varied the TBF queue size, burst size, and token generation rate:

- Q_{config} was tested as small packet-sized queues, approximately $4p, 8p, 16p, 32p, 64p$, where $1p = 1460$ bytes;
- $B \in \{5840, 12500, 25000, 50000, 100000\}$ bytes;
- $R_{\text{tbf}} \in \{500, 750, 1000\}$ Mbit/s.

For each configuration, four strategies were compared:

- uncapped Reno-like behavior using `my_cca` with `cwnd_limit = 0`;
- an empirical cap found from physical measurements;
- Friedli's $\text{BDP} + Q_{\text{config}} - \text{MSS}$ cap, referred to here as the queue-aware BDP formula;
- Friedli's simpler $\text{BDP} + \text{MSS}$ cap, referred to here as the BDP-plus-one-MSS formula.

The empirical loss threshold was 0.20 for the two smallest queue sizes and 0.05 for larger queues. This change should be kept in mind when comparing empirical choices across queue-size groups.

4.2. Empirical Cap Selection

The empirical cap was not another closed-form formula. It was a measured reference point used to understand what window the physical setup could actually sustain for each TBF configuration. The first step was to run the same configuration with normal Reno-like behavior, using `my_cca` with `cwnd_limit = 0`. From this uncapped run, the analysis extracted the post-slow-start `cwnd` samples before the connection entered a loss state. The useful samples are the top points of the Reno sawtooth: they indicate how far Reno can grow before the TBF and TCP recovery mechanisms push it down again. Averaging these top `cwnd` values gives an initial estimate of the largest safe window for that configuration.

The second step was to rerun the transfer with explicit server-side caps. The search started near the uncapped Reno top-`cwnd` estimate and then gradually decreased the configured `cwnd_limit`. For each capped run, the pipeline recorded throughput and a loss rate computed from the kernel phase labels:

$$\text{loss rate} = \frac{\#\{\text{loss_recovery rows}\}}{\#\{\text{logged phase rows}\}}.$$

The ideal target is zero loss-recovery rows. When several candidates were effectively loss-free, the higher-throughput candidate was preferred; when none was completely loss-free, the candidate with the smallest loss rate was used. A stricter diagnostic also considered `fast_retransmit`. The empirical cap is therefore the cap found by physical measurement: it is the value that removes most of the loss-recovery behavior while keeping the transfer close to the best observed throughput. This empirical value is not meant to be deployable by itself; it is the ground truth used to judge whether the formulas from the simulation papers remain accurate on the Linux testbed.

4.3. Early Sanity Check: Large Queue

The first stable setup used $R_{\text{tbf}} = 1 \text{ Gbit/s}$, burst size 1 MB, and $Q_{\text{config}} = 50000 \text{ B}$. With RTT around 0.4 ms, the BDP is about 50000 B, so the original formula predicts a cap close to 66–67 MSS-sized segments. This experiment is documented in the project notes in `client_server/README` and summarized again in `client_server/report1/optimalCWND.md`. The measured result matched the prediction: `cwnd_limit` around 66–67 produced a flat and stable trace.

However, the throughput improvement over Reno was small: roughly 9.20 s for the capped runs versus about 9.51 s on average for Reno-like runs when sending 1 GiB. This is not a contradiction of the ns-3 work. Friedli’s results already showed that once the TBF queue reaches approximately the BDP, Reno can often keep the outgoing link busy despite its sawtooth. The physical setup first validated this large-queue regime before moving to smaller queues.

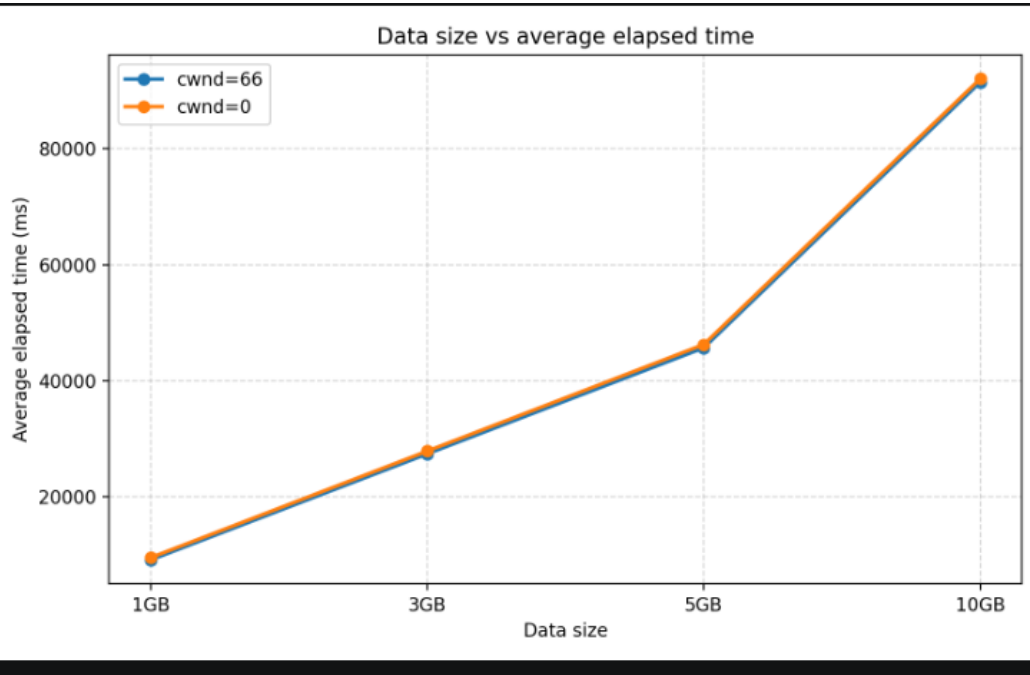


Figure 6: Early large-queue sanity check comparing uncapped Reno-like behavior with a `cwnd` cap near 66 segments.

4.4. Sweep Results

Across all 75 queue/burst/rate points, the empirical cap was the most reliable high-throughput choice. The generated analysis in `client_server/report4` reports the following aggregate metrics:

Strategy	Mean throughput (MiB/s)	Best-throughput wins
TCP Reno	65.34	6
Empirical cap	79.88	51
BDP + $Q_{\text{config}} - \text{MSS}$	65.01	10
BDP + MSS	65.41	8

 Table 1: Aggregate result over the 75 physical TBF configurations parsed in **report4**.

The gain is concentrated in small queues. Table 2 shows the average empirical-over-Reno throughput gain by queue size. The queue sizes are reported in packet units to make the small-buffer regime easier to see.

Queue size	Mean empirical gain over Reno (MiB/s)
4.3p	47.47
8.6p	20.53
17.1p	1.62
34.2p	0.24
68.5p	2.88

Table 2: The benefit of a window cap is largest when the TBF queue is small.

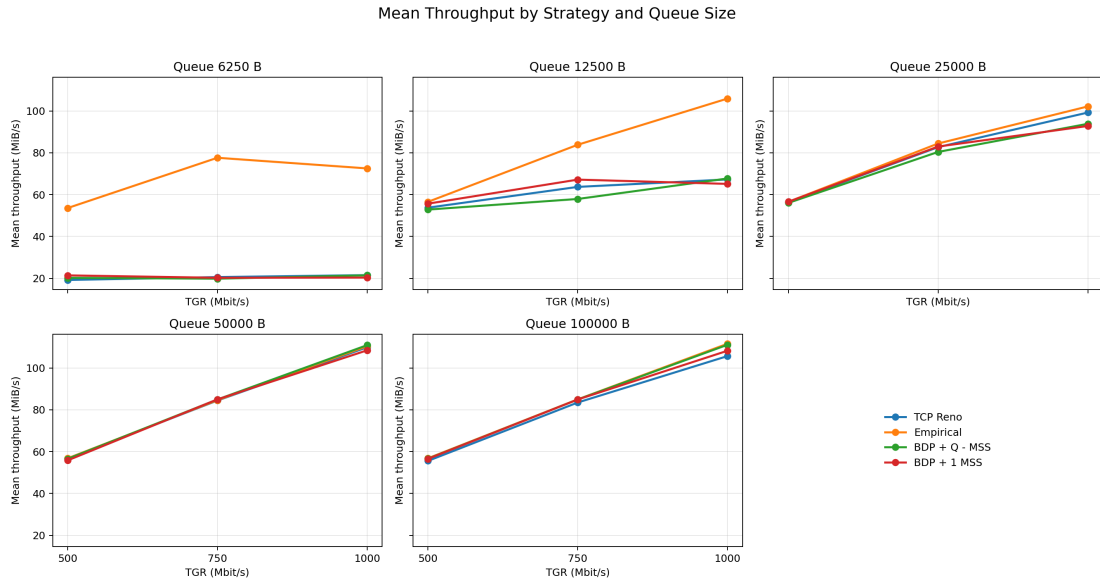


Figure 7: Mean throughput by strategy and queue size. Empirical capping dominates in small queues, while all strategies converge for larger queues.

The largest positive empirical-over-Reno case occurred with $Q_{\text{config}} = 6250$ B, about 4.3p, burst size 12500 B, and $R_{\text{tbf}} = 750$ Mbit/s: empirical capping reached 79.46 MiB/s while Reno reached only 10.47 MiB/s. The largest negative case was small: with $Q_{\text{config}} = 25000$ B, burst size 25000 B, and $R_{\text{tbf}} = 1000$ Mbit/s, empirical capping reached 104.69 MiB/s while Reno reached 108.95 MiB/s.

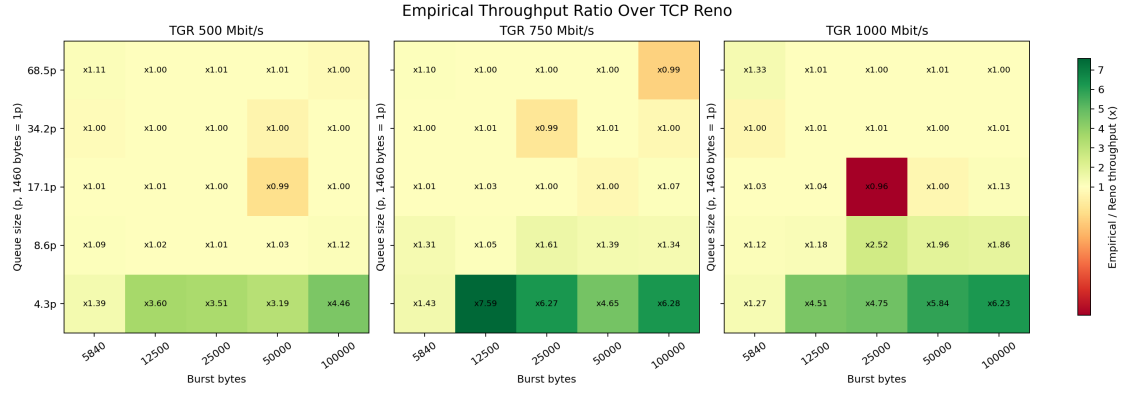


Figure 8: Empirical-over-Reno throughput ratio. A value such as $x5$ means the empirical cap achieved five times the Reno throughput for the same TBF configuration. The high-gain region is concentrated at small queues and high token generation rates.

One representative high-gain case is $R_{\text{tbf}} = 1$ Gbit/s, a queue of about $4p$, and a burst size of 50000 B. In this case Reno repeatedly overshoots the small TBF queue, while the empirical cap keeps the sender closer to the usable window. If a plotted trace shows `cwnd` dropping from the Linux initial value 10 to a value below 10 while still labeled `slow_start`, this is not necessarily contradictory. When the configured cap is smaller than the initial `cwnd`, the first ACK callbacks in `my_cca` clamp `cwnd` down to the cap even though the phase is still slow start because `cwnd < ssthresh`. In an uncapped small-queue Reno run, an early packet loss can also reduce `cwnd` before the logging phase label has moved into steady congestion avoidance.

4.5. Why the Original Formula Overestimated

The original algorithm assumes that the configured TBF queue can be used as extra in-flight capacity:

$$W = \text{BDP} + Q_{\text{config}} - \text{MSS}.$$

This is accurate when the packet arrival rate into the TBF is close to the token generation rate. In the physical setup, however, the server application sends in bulk and the TCP stack can enqueue into the TBF faster than the qdisc drains. Let R_{arrival} be the short-window rate at which bytes arrive at the TBF enqueue path. When $R_{\text{arrival}} > R_{\text{tbf}}$, the queue grows during one RTT by approximately:

$$Q_{\text{tbf}} = \max(0, (R_{\text{arrival}} - R_{\text{tbf}})RTT_{\text{base}}).$$

This queue growth is not free buffer space. It is a burst penalty that must be subtracted from the configured queue:

$$Q_{\text{effective}} = \max(0, Q_{\text{config}} - Q_{\text{tbf}}).$$

The revised estimate is therefore:

$$W = \left\lfloor \frac{\text{BDP} + Q_{\text{effective}} - \text{MSS}}{\text{MSS}} \right\rfloor.$$

When $Q_{\text{tbf}} > Q_{\text{config}}$, even one RTT of enqueue/drain mismatch does not fit into the queue. The project used a conservative reduced-BDP branch:

$$\text{BDP}_{\text{usable}} = \text{BDP} \cdot \min\left(1, \frac{Q_{\text{config}}}{Q_{\text{tbf}}}\right), \quad W = \left\lfloor \frac{\text{BDP}_{\text{usable}} - \text{MSS}}{\text{MSS}} \right\rfloor.$$

This model explains why $\text{BDP} + Q_{\text{config}} - \text{MSS}$ was too aggressive in many small-queue physical runs. For example, with $R_{\text{tbf}} = 500$ Mbit/s, $\text{RTT} = 0.4$ ms, and $Q_{\text{config}} = 50000$ B, the BDP is 25000 B. If

packets arrive at the TBF near 1 Gbit/s, then the burst penalty is also around 25000 B. The effective queue is therefore about 25000 B, and the expected cap becomes roughly:

$$\left\lfloor \frac{25000 + 25000 - 1460}{1460} \right\rfloor = 33,$$

not 50. This matches the empirical region observed in the physical tests.

This is the main reason the two ns-3 window rules do not directly transfer to every physical configuration. In many small-queue runs, both the empirical `cwnd` and the Reno `top-cwnd` are smaller than the values predicted by $\text{BDP} + Q_{\text{config}} - \text{MSS}$, and sometimes also smaller than the simpler BDP-plus-one-MSS rule. The missing variable is not the token generation rate; it is the short-term enqueue rate into the TBF. When the empirical `cwnd` is used in reverse to infer what R_{arrival} must have been, the resulting arrival-aware estimate is much closer to the measured cap. This supports the interpretation that the formulas need an arrival-pressure term in the physical setup.

4.6. Remaining Model Mismatch

The arrival-aware correction is an improvement, but not a complete model. In the very small queue cases, especially $Q_{\text{config}} = 6250$ B, about $4.3p$, the reduced-BDP branch can underpredict the best empirical `cwnd`. For example, some 500 Mbit/s runs were best around `cwnd` 7 while the conservative formula predicted smaller values. The reason is that R_{arrival} is not a stable whole-transfer constant. The relevant value is a short burst into the TBF over approximately one base RTT, and this value depends on offloads, pacing, qdisc behavior, and the current TCP phase.

It is important to separate two different problems that looked similar in the early notes. The full kernel ring buffer was a measurement problem: it made the beginning of the `cwnd` trace disappear, so the plots could look as if TCP started from an impossible state. After the background `dmesg` extractor was added, this artifact was greatly reduced. It was not the network mechanism that caused the throughput plateau.

The plateau and collapse in the small-queue experiments came from the data path itself. A bulk-sending server can deliver packets to the TBF enqueue path faster than the TBF token rate. During these short bursts, the configured qdisc queue is partly consumed before it can act as useful BDP-plus-queue headroom. If the cap is too large, this pressure pushes the flow into loss recovery; if the cap is small enough, the sender stays near the TBF rate without repeated recovery. This is why R_{arrival} matters, and why the same R_{tbf} and BDP can lead to different practical caps when the sender pacing and qdisc behavior change.

One possible follow-up would be to modify the server so that R_{arrival} is intentionally controlled. For example, the server could expose `SO_MAX_PACING_RATE` as an experiment parameter, add application-level pacing between send calls, or use `TCP_NOTSENT_LOWAT` to limit how much unsent data accumulates in the TCP stack. This would make it possible to run a cleaner experiment with $R_{\text{arrival}} \approx R_{\text{tbf}}$, and then with known ratios such as $1.25R_{\text{tbf}}$ or $2R_{\text{tbf}}$. The reason this was not the main experiment is that the target deployment should work with an ordinary bulk sender, not only with a cooperative sender that has already been paced to the bottleneck rate. Pacing the server would be useful for isolating the model, but it would also hide exactly the burst pressure that made the physical setup differ from ns-3.

5. Experiment 2: Receiver-Side RWND Control

5.1. Motivation

Directly capping server-side `cwnd` is useful for controlled experiments, but it is not the final deployment model. For a downstream user, the limiter is usually on the path toward the receiver, and the receiver cannot assume that the remote server will install a custom congestion-control module. The natural

receiver-side control knob is therefore `rwnd`. Since TCP sends according to $\min(\text{cwnd}, \text{rwnd})$, the client can attempt to impose the same effective cap by advertising a smaller receive window.

5.2. Method

For each selected configuration, the server was run with `my_cca` but with `cwnd_limit = 0`, so the server-side cap was disabled. The client then ran:

```
python3 Client/client.py -rwnd-segments <empirical-cwnd>.
```

The client script converts the segment count to bytes using MSS 1460 B, sets `SO_RCVBUF`, and sets `TCP_WINDOW_CLAMP`. This forces the sender's effective window to be limited by the advertised receive window.

5.3. Results

The receiver-side cap reproduced the sender-side empirical behavior in most cases. Table 3 shows representative examples from `report3/RWND.md`.

Queue	R_{tbf} (Mbit/s)	Empirical <code>cwnd</code>	Sender cap (MiB/s)	Client <code>rwnd</code> cap (MiB/s)
4.3p	500	7	56.94	56.23
8.6p	500	9	56.58	56.60
17.1p	750	18	84.34	85.27
34.2p	500	33	56.79	56.75
68.5p	1000	102	111.98	111.70

Table 3: Representative receiver-side `rwnd` tests. Advertising the empirical window from the client closely matches sender-side `cwnd` capping.

Some small-window cases are sensitive: for example, increasing `rwnd` from 9 to 10 segments in the 750 Mbit/s, about 4.3p queue case caused a large drop-rate increase and throughput collapse. This confirms that receiver-side control is powerful enough to reproduce the cap, but also that the selected cap must be accurate.

6. Experiment 3: Estimating the Token Generation Rate

6.1. Relation to Policing-Rate Inference

The window-comparison experiments above assume that the TBF parameters are known. That is useful for validation: it lets us ask whether a good window exists when R_{tbf} , queue size, and burst size are controlled. A real receiver-side system, however, should not rely on being given those parameters. The receiver must estimate the limiter rate from the traffic it observes, and then advertise an `rwnd` cap derived from that estimate.

The natural final system would combine the two previous papers as follows:

1. the receiver infers R_{tbf} from arrivals, using a method such as cumulative-byte differentiation;
2. the receiver estimates RTT;
3. the receiver computes a target window;
4. the receiver advertises that target through `rwnd`.

The physical experiments support steps 3 and 4: once a reasonable target window is known, the receiver can enforce it with `rwnd`. The remaining challenge is step 1. Inferring the output policing rate tells us the long-term bottleneck rate. It does not tell us how quickly the sender is enqueueing into the TBF, nor how much of the configured qdisc queue is already consumed by burst pressure.

6.2. Cumulative Arrival at the Receiver

The cumulative-arrival method from Rimkevicius’s work is still useful for estimating the bottleneck rate under balanced conditions. In the physical setup, the receiver records cumulative received bytes over time and estimates the slope after the initial transient. When the burst size is not too large and the flow is already constrained near the token generation rate, this slope approximates R_{tbf} well enough to recover the server-side BDP used in the window formulas.

However, the empirical `rwnd` needed in the physical setup also depends on R_{arrival} . The receiver only observes delivered bytes after the TBF; it does not directly observe how fast the sender tried to enqueue packets before the TBF.

This explains why a receiver can estimate the BDP but still fail to choose the empirical receive window in aggressive bulk-send cases. The hidden variable is not the delivered rate, but the enqueue pressure at the qdisc.

6.3. Receiver-Assisted Enqueue-Rate Estimation

7. Limitations

The project validates the core idea in a real Linux setup, but several limitations remain.

First, the bottleneck was implemented as a Linux server-egress qdisc. Real ISP policing is usually downstream of the sender and may be ingress from the user’s perspective. Linux can emulate ingress shaping through `ifb`, but early tests with `ifb0` did not expose a clean backlog signal, so the stable experiments used server egress.

Second, RTT was very small and difficult to control precisely in the physical LAN. Some experiments attempted to add `netem` delay, but queue placement and interaction between `netem` and TBF changed the results. More complete validation should repeat the sweep across controlled RTTs.

Third, the experiments focus on Reno-like behavior through `my_cca`. Cubic and BBR were studied in ns-3 but not fully reproduced in the physical setup.

Fourth, the empirical cap search used loss thresholds that differed by queue range. This was practical during experimentation, but a final system should define a single objective function balancing throughput, latency, and loss.

Finally, the TBF queue, NIC driver, offload settings, and Linux pacing interact in ways that are hard to reduce to one closed-form formula. Disabling TSO and GSO improved interpretability, but it also means the setup is not exactly the default production Linux network path.

Future work should connect the window-control mechanism with a bottleneck-detection system. A Wehe-like detector [5] could first determine whether the endpoint ISP is the likely bottleneck; once that is known, the receiver-side method in this project could be used to choose an `rwnd` that uses the available queue space more efficiently. The harder open problem is R_{arrival} . If the enqueue rate is the reason the simulation formulas do not directly apply, then a practical receiver-side system needs either a way to estimate this hidden variable or a conservative control loop that can adapt without observing it directly. This is especially difficult in the small-queue regime, where the empirical window can be smaller than the BDP and the receiver sees only the already-shaped output stream.

8. Conclusion

References

- [1] Franziska Friedli, *Revisiting Traffic Congestion Control Protocols at the Last Mile*, EPFL NAL Semester Project, 2025.
- [2] Martynas Rimkevicius, *Inference of Traffic Differentiation Policing Rate*, EPFL NAL Semester Project, 2025.
- [3] Somaya Arianfar, *TCP's Congestion Control Implementation in Linux Kernel*, Aalto University, 2012.
- [4] Tobias Flach et al., *An Internet-Wide Analysis of Traffic Policing*, ACM SIGCOMM, 2016.
- [5] Fangfan Li et al., *A Large-Scale Analysis of Deployed Traffic Differentiation Practices*, ACM SIGCOMM, 2019.
- [6] Linux manual page, *tc(8)*, <https://man7.org/linux/man-pages/man8/tc.8.html>.
- [7] Linux manual page, *tc-tbf(8)*, <https://man7.org/linux/man-pages/man8/tc-tbf.8.html>.
- [8] Linux manual page, *dmesg(1)*, <https://man7.org/linux/man-pages/man1/dmesg.1.html>.
- [9] Linux kernel documentation, *Message logging with printk*, <https://docs.kernel.org/core-api/printk-basics.html>.